Box-Office Revenue Prediction Using Elastic Net Regression

A Project Report Submitted in Partial Fulfillment of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE (AIDS)
AND
COMPUTER SCIENCE AND INFORMATION TECHNOLOGY (CSIT)

By

SUDANI GOPALA KRISHNA SRINIVASA RAO 2520080032 (AIDS)

SHANMUKHA SAGAR 2520090050 (CSIT)

Under the Esteemed Guidance of

Dr. Lakshmi Prasanna

[Designation and Department]

K L (Deemed to be) University

[Institution address, if required]

Academic Year: [Add academic year]

DECLARATION

We hereby declare that the project report entitled **"Box-Office Revenue Prediction Using Elastic Net Regression"** is the record of work prepared by us for academic purposes. The report describes the project implementation, dataset, algorithms, machine-learning workflow, application, evaluation considerations, and known limitations. The work is submitted in partial fulfillment of the academic requirements of the relevant programme and institution.

We further declare that the project content is presented to the best of our knowledge and that sources used for technical references are acknowledged in the References chapter. Any final institutional declaration, course code, section, or team designation should be completed according to the department's submission rules.

SUDANI GOPALA KRISHNA SRINIVASA RAO - 2520080032

SHANMUKHA SAGAR - 2520090050

Place: _____ Date: _____

CERTIFICATE

This is to certify that the project report entitled **"Box-Office Revenue Prediction Using Elastic Net Regression"** is submitted by SUDANI GOPALA KRISHNA SRINIVASA RAO (2520080032, AIDS) and SHANMUKHA SAGAR (2520090050, CSIT) in partial fulfillment of the academic requirements of [Programme / Course / Institution].

The project integrates a movie-data analytics application, classical data-structures-and-algorithms implementations, and a supervised Elastic Net regression workflow. This certificate page is provided in the format of the supplied sample and requires review and signature by the authorized academic guide and department.

Signature of the Guide	Signature of the Course Coordinator	Signature of the HOD
_____	_____	_____

Name: [Guide]

Department: [Department / Institution]

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Lakshmi Prasanna, our course coordinator, and faculty members for their guidance, feedback, and support during the development of this project. Their suggestions helped us connect the machine-learning workflow with the data structures and algorithms used for movie search and recommendations.

We also thank our department and institution for providing the academic environment and resources required to complete this work. We acknowledge the maintainers of the open-source libraries and documentation referenced in this report, including Python, pandas, scikit-learn, Streamlit, NumPy, and Java.

Finally, we thank our families and classmates for their encouragement and assistance throughout the project.

SUDANI GOPALA KRISHNA SRINIVASA RAO - 2520080032

SHANMUKHA SAGAR - 2520090050

ABSTRACT

The Box-Office Revenue Prediction and Movie Analytics System is an academic application that combines movie catalogue exploration, title search, metadata-based recommendations, and revenue estimation. It integrates a Python and Streamlit user interface, a scikit-learn machine-learning pipeline, and a Java data-structures-and-algorithms engine with Python fallbacks. The current source dataset contains 5,000 movie records and 13 columns covering worldwide and regional gross, release year, genres, rating, vote count, original language, and production countries.

The predictive component treats worldwide gross as a continuous target and uses Elastic Net regression, which combines L1 and L2 regularization. Numeric values are standardized and categorical values are one-hot encoded within a scikit-learn pipeline. The baseline model uses the fields currently present in the dataset: year, genres, rating, vote count, original language, and production countries. The project also implements KMP and Rabin-Karp title search, edit-distance typo matching, suffix-array similarity, graph-based metadata recommendations, Ford-Fulkerson and Edmonds-Karp maximum-flow demonstrations, and greedy/randomized selection heuristics.

A key scope limitation is that the supplied CSV does not contain production budget, runtime, cast, director, or release-season features. Consequently, this report does not claim that an enriched model using those variables has been trained. Rating and vote count may also be observed after a film is released, so the current model is best understood as a historical estimation and educational analytics tool, not a validated pre-release forecasting service. The system demonstrates how data preparation, model fitting, serving, search, analytics, and responsible interpretation connect in one project.

Keywords: Box office, Elastic Net, regression, movie analytics, KMP, Rabin-Karp, edit distance, graph algorithms, Streamlit, scikit-learn.

INDEX

CHAPTER 1: INTRODUCTION 7

CHAPTER 2: SYSTEM REQUIREMENTS 8

CHAPTER 3: TECHNOLOGY STACK 9

CHAPTER 4: SYSTEM ARCHITECTURE 10

CHAPTER 5: DESIGN 11

CHAPTER 6: IMPLEMENTATION 13

CHAPTER 7: FEATURES AND USER WORKFLOW 16

CHAPTER 8: TESTING AND MODEL EVALUATION 17

CHAPTER 9: DEPLOYMENT AND REPRODUCIBILITY 18

CHAPTER 10: CHALLENGES, LIMITATIONS, AND RESPONSIBLE USE 19

CHAPTER 11: FUTURE ENHANCEMENTS 20

CHAPTER 12: CONCLUSION 21

CHAPTER 13: REFERENCES 22

CHAPTER 14: APPENDICES 23

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

The film industry produces extensive historical information about titles, genres, release years, audience ratings, and box-office performance. Analysts need to find relevant titles, compare similar films, and examine revenue patterns without manually searching many files. A combined analytics application can make this exploration more accessible while providing a reproducible modelling workflow.

This project brings together catalogue search, descriptive analytics, metadata-based recommendations, and a supervised regression model. It is designed as a learning and decision-support prototype. It is not a commercial forecasting platform and does not guarantee a film's future revenue.

1.2 Problem Statement

Given a movie record and historical catalogue, the system should allow users to locate titles efficiently, inspect relevant metadata, identify related movies, and estimate worldwide gross from supported inputs. The implementation must also explain how raw data becomes model-ready features and how the trained pipeline is loaded for application use.

1.3 Scope of the Project

- Movie-data loading, cleaning, schema normalization, and exploratory summaries.
- Elastic Net regression for the continuous worldwide-gross target.
- Exact, approximate, and similarity-based title search.
- Metadata graph recommendations and algorithm demonstrations.
- A Streamlit interface for prediction, search, recommendations, model comparison, and analytics.
- Reproducible setup and training/evaluation scripts.

1.4 Objectives

- Build a connected movie analytics application using the supplied dataset.
- Implement a regularized regression pipeline and document its assumptions.
- Demonstrate string algorithms, dynamic programming, graph flow, approximation, and randomized selection.
- Present historical data and model outputs in a usable interface.
- Identify data limitations and prevent unsupported pre-release prediction claims.

1.5 Importance and Target Users

The project provides a single place to explore historical movie records, test algorithmic search, and connect machine-learning concepts to a working application. Its primary audience is students learning applied machine learning, data structures, and production-style project organization; secondary users include analysts exploring historical film performance.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

A modern computer with a multi-core processor, at least 8 GB RAM, and several gigabytes of free storage is suitable for local development and demonstration. The current dataset is modest; the saved model and Java sources do not require specialized hardware or a GPU.

2.2 Software Requirements

- Windows 10/11, Linux, or macOS.
- Python environment compatible with the packages listed in requirements.txt.
- A modern browser for the Streamlit interface.
- Java Development Kit when running the Java DSA engine; Python fallbacks are available for supported operations.
- Visual Studio Code, IntelliJ IDEA, Eclipse, or Jupyter Notebook for development (optional).

2.3 Development Tools and Libraries

Python is used for data processing, modelling, analytics, and the interface. pandas and NumPy support tabular data operations; scikit-learn supplies preprocessing, regression, model selection, and metrics; Streamlit creates the interactive UI; Matplotlib and Seaborn support plotting. Java implements the algorithm engine. CSV files store the dataset and derived tables.

CHAPTER 3: TECHNOLOGY STACK

3.1 Python Application Layer

The root launcher app/app.py starts the main Streamlit application at src/app/app.py. The UI coordinates page-level flows and calls data, prediction, and DSA modules. A local application is launched from the project root with the command documented in Appendix B.

3.2 Data and Machine-Learning Libraries

pandas loads and transforms CSV records; NumPy supplies numeric operations; scikit-learn provides ColumnTransformer/Pipeline components, StandardScaler, OneHotEncoder, ElasticNet, train/test splitting, cross-validation, search, and evaluation metrics. Plotting tools are used for exploratory charts and actual-versus-predicted diagnostics.

3.3 Java DSA Engine

java/src/main/java/MovieDsaEngine.java implements search and graph algorithms. src/movie_dsa.py provides Python implementations and a bridge to the Java engine when a Java runtime is available. This dual path allows core functionality to remain usable when Java is unavailable while retaining the Java implementation for demonstration.

3.4 Storage and Versioned Artifacts

The original CSV is stored in data/movies.csv. Cleaned and model-ready data are written below data/processed/. Trained scikit-learn pipelines are stored in models/. Graphs and profile outputs are kept in graphs/. The optional data/enriched/movies_enriched.csv path is supported by the feature preparation workflow but is not present in the current project dataset.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture

The application separates data preparation, feature construction, model training, algorithmic catalogue operations, and presentation. The UI is a local consumer of the model artifact and movie catalogue; the current codebase is not configured as a hosted REST service.

DATA SOURCE	PREPARATION	MODEL / ALGORITHMS	USER INTERFACE
data/movies.csv	clean_data.py prepare_features.py	Elastic Net pipeline Java/Python DSA engine	Streamlit app Search / Predict / Analytics

Figure 4.1. Project data and application flow.

4.2 End-to-End Prediction Flow

A user selects or enters movie attributes in the application. The app validates available fields and prepares the feature record using the same names and types expected by the model. The serialized scikit-learn pipeline applies numeric scaling and categorical encoding before generating the worldwide-gross estimate. The UI displays the estimate alongside context such as historical analytics. The model pipeline must remain responsible for preprocessing at inference time to reduce training-serving mismatch.

4.3 Training-Serving Boundary

A frequent production bug occurs when training and serving apply different cleaning, category handling, or scaling. This is known as training-serving skew. In this project, preprocessing is packaged with the estimator in a scikit-learn pipeline so the same transformations can be reused. A future production version should add explicit schema validation, artifact versioning, data freshness checks, latency logging, and monitoring.

CHAPTER 5: DESIGN

5.1 Dataset Description

The supplied data/movies.csv file contains 5,000 records and 13 columns spanning release years 2000-2024. Worldwide gross ($Worldwide) is the prediction target. Movie title supports search and display but is not used as a baseline predictor. Domestic and foreign grosses and shares are retained as descriptive fields, not model inputs.

Field	Description	Baseline role
Rank	Source-list position	Excluded
Release Group	Movie title	Search/display
$Worldwide	Worldwide gross revenue	Target
$Domestic / Domestic %	Domestic gross and share	Excluded
$Foreign / Foreign %	Foreign gross and share	Excluded
Year	Release year	Predictor
Genres	Genre labels	Predictor
Rating	Audience rating	Predictor
Vote_Count	Audience vote count	Predictor
Original_Language	Language metadata	Predictor
Production_Countries	Production-region metadata	Predictor

Table 5.1. Source fields and their current modelling role.

5.2 Data-Quality Profile

The available profile shows missing values in several metadata fields: 178 genre records, 170 ratings, 170 vote counts, 170 original-language values, and 200 production-country values. A total of 203 rows have at least one missing listed field. The source has repeated title groups that are not necessarily exact duplicate records; repeated titles should not be removed without review.

Statistic	Observed value
Worldwide gross minimum	about $1.66 million
Worldwide gross median	about $48.45 million
Worldwide gross mean	about $119.21 million
Worldwide gross maximum	about $2.80 billion
Vote count present	4,830 of 5,000

Table 5.2. Summary values from the supplied dataset profile.

5.3 EDA Interpretation

Mean revenue is substantially greater than median revenue, indicating a long right tail and influential high-grossing films. MAE should be reported alongside RMSE because RMSE penalizes large residuals more heavily. Missingness should be handled within a reproducible preparation workflow. A log-target experiment may help with skew, but its predictions must be inverse-transformed and evaluated on the original revenue scale.

CHAPTER 6: IMPLEMENTATION

6.1 Data Ingestion and Schema Normalization

The original movie file remains the source of record. src/data_schema.py contains canonical names and aliases so headers from compatible enriched files can map to common feature names. This reduces brittle column-name assumptions across cleaning, preparation, and training scripts.

6.2 Cleaning Workflow

- Remove rows that are exact duplicates across every field; repeated titles alone are retained.
- Convert rating strings such as "7.5/10" into numeric ratings.
- Convert revenue, percentage, year, rating, and vote-count values to numeric types.
- Impute missing year, rating, and vote count with median values; replace missing genre, language, and country labels with "Unknown".
- Remove records without the worldwide-revenue target.
- Write the cleaned table to data/processed/movies_cleaned.csv.

6.3 Feature Preparation and Transformation

src/prepare_features.py chooses the enriched file if present; otherwise it uses the cleaned baseline. It selects canonical model fields and writes data/processed/model_data.csv. Numeric inputs are standardized using StandardScaler, while categorical inputs are one-hot encoded with unknown categories ignored. The transformations are embedded in the full estimator pipeline.

6.4 Point-in-Time Correctness

For a genuine pre-release forecast, all predictors must be available at the intended decision time. Rating and vote count may accumulate after release and can leak post-release information into a model presented as pre-release. A future version should define the forecast cutoff and use only information available by that cutoff, with a time-based holdout evaluation.

6.5 Machine-Learning Model

6.5.1 Learning Problem and Model Choice

This is supervised learning because each training record has input features and a known target. It is a regression problem because worldwide gross is a continuous numeric quantity. It is not a classification task, an unsupervised clustering task, or a reinforcement-learning system. Elastic Net is a linear regression estimator combining L1 and L2 penalties.

6.5.2 Elastic Net Objective

Elastic Net minimizes prediction error while penalizing coefficient magnitude. The L1 component can shrink some coefficients to zero, encouraging sparse feature effects; the L2 component stabilizes coefficients when predictors are correlated. This combination is useful for one-hot encoded metadata with overlapping signals. The strength of regularization is controlled by alpha, and l1_ratio sets the L1/L2 mixture.

```
Objective: squared-error term + alpha * [l1_ratio * L1 penalty
```

```
                                + (1 - l1_ratio) * L2 penalty]
Baseline configuration: alpha = 0.1, l1_ratio = 0.5
```

6.5.3 Training and Tuning

src/train_model.py fits the baseline pipeline with an 80/20 random train/test split and stores the fitted artifact under models/. src/tune_model.py searches candidate hyperparameters. src/train_log_model.py provides an optional log-target experiment, and src/train_enriched_model.py is an optional path that requires a correctly prepared enriched dataset. These alternatives should be compared using a fixed validation plan before selecting a final model.

6.5.4 Model Artifact and Prediction

src/predict_revenue.py provides a console prediction workflow. The Streamlit application loads the saved model pipeline and supplies a feature record with compatible canonical columns. Packaging preprocessing and the estimator together helps ensure the same transformations are applied during training and prediction.

6.6 Data Structures and Algorithms Implementation

6.6.1 Exact Search: KMP and Rabin-Karp

Knuth-Morris-Pratt preprocesses a pattern into a prefix/failure table. When a mismatch occurs, it reuses the known matched prefix rather than restarting from the beginning; its search time is linear in the text and pattern lengths after preprocessing. Rabin-Karp compares rolling hashes over candidate windows and confirms character equality when hashes match. It is useful for demonstrating substring search and can support multiple-pattern extensions.

6.6.2 Approximate Matching: Edit Distance and Suffix Arrays

Edit distance uses dynamic programming to compute the minimum insertions, deletions, and substitutions needed to transform one title into another. This supports typo-tolerant suggestions. Suffix-array matching orders suffixes and uses common-prefix comparisons to identify similar title fragments. Similarity results are retrieval aids and should be interpreted as string resemblance, not semantic similarity.

6.6.3 Graph-Based Recommendations and Flow

Movie metadata are represented as shared-feature links. A graph can connect titles that share genre or other supported metadata; candidate recommendations are shortlisted using maximum-flow routines implemented with Ford-Fulkerson and Edmonds-Karp. In this project, the graph is a metadata-connectivity demonstration. It is not a causal measure of actor/director influence and should not be described as a learned recommender.

6.6.4 Approximation and Randomized Choices

A greedy coverage heuristic selects candidates that add useful shared metadata while avoiding redundant recommendations. Seeded randomized tie-breaking provides repeatable choice among equal candidates. These techniques improve practical selection and demonstrate algorithmic paradigms; they do not guarantee a globally optimal recommendation set.

6.6.5 Complexity and Practical Trade-Offs

KMP and Rabin-Karp support efficient scans of title text; dynamic-programming edit distance is quadratic in the lengths of two strings in its standard form; suffix arrays require preprocessing and additional storage; graph maximum flow depends on the graph representation and residual-edge traversal. For the current local catalogue, clarity and algorithmic demonstration are central goals. Larger collections would benefit from indexing and profiling.

CHAPTER 7: FEATURES AND USER WORKFLOW

7.1 Movie Search and Recommendations

The search workflow lets a user find a movie by title using exact matching and typo-tolerant approaches. The recommendations workflow uses shared metadata and graph connectivity to return related titles. Users can explore why records are grouped by metadata rather than interpreting a match as a prediction of taste.

7.2 Revenue Prediction

The prediction interface accepts fields supported by the active model pipeline and returns an estimated worldwide gross. The currently available baseline uses year, genres, rating, vote count, original language, and production countries. The UI must not imply that budget, runtime, cast popularity, director popularity, or release season are active predictors unless an enriched dataset has been supplied and the corresponding model has been trained.

7.3 Analytics and Model Comparison

The analytics view summarizes the movie dataset and displays distributions and relationships supported by the data. Model-comparison functions allow users to inspect candidate model outputs. Saved graphs include an actual-versus-predicted evaluation plot. Exact scores should be taken from a fresh evaluation run and reported with the split protocol, target transformation, and metric definitions.

7.4 Application Entry Points

The stable launcher is app/app.py; the main UI is src/app/app.py. The separate src/analytics.py module and console scripts provide supporting analysis and model workflows. README.md documents the quick-start command and points to the longer project documentation.

CHAPTER 8: TESTING AND MODEL EVALUATION

8.1 Data and Pipeline Checks

Useful checks include confirming expected CSV columns, row counts, numeric conversions, missing-value treatment, model input names, and the ability to serialize and reload the full pipeline. Search implementations should be compared on representative exact and misspelled queries, while recommendation behavior should be checked for valid catalogue results and deterministic tie-breaking when a seed is used.

8.2 Regression Metrics

MAE gives the average absolute revenue error in the target's currency units. RMSE is more sensitive to large errors. R-squared describes variance explained relative to a mean baseline but can be negative on a test set. MAPE can be unstable when actual revenues are small; a metric should be chosen to match the decision context. The project evaluation script is src/evaluate_model.py and can regenerate the actual-versus-predicted plot.

8.3 Split Discipline and Leakage

A test set must remain unused during feature selection, hyperparameter selection, and repeated model comparison. The baseline uses a random 80/20 split, which is useful for an instructional baseline but does not simulate predicting future releases. A time-based split is more appropriate for temporal forecasting. Rating and vote count also require a clearly stated prediction cutoff.

8.4 Results Reporting

This report avoids publishing numeric model-performance claims that were not freshly verified from a documented evaluation run. Before final academic submission, run the training/evaluation workflow and add the resulting MAE, RMSE, and R-squared along with the exact test split, model artifact, and date of execution. The existing plot alone is not sufficient evidence for a score or generalization claim.

CHAPTER 9: DEPLOYMENT AND REPRODUCIBILITY

9.1 Local Application Deployment

The current delivery target is a local Streamlit application. From PowerShell, change to the project directory and start the UI using the existing virtual-environment Python executable. If Windows denies access to that environment, create a separate environment rather than deleting or modifying the existing one.

```
cd C:\Users\gopal\OneDrive\Desktop\BoxOfficeRevenuePrediction
.\venv\Scripts\python.exe -m streamlit run .\app\app.py
```

9.2 Reproducible Data and Model Workflow

The documented sequence is: inspect the source CSV; run cleaning; prepare model features; train the selected baseline or experiment; evaluate the saved model; then run the application. Record the dataset version, feature list, preprocessing choices, split method, hyperparameters, metrics, and artifact path for each experiment. Do not overwrite the original CSV with processed data.

9.3 Production Readiness

The present project runs locally and does not configure authentication, a REST endpoint, cloud hosting, a feature store, production monitoring, or automated retraining. A production service would require request validation, secure artifact loading, endpoint and latency checks, logging, drift and performance monitoring, access control, and a tested retraining process.

CHAPTER 10: CHALLENGES, LIMITATIONS, AND RESPONSIBLE USE

10.1 Data Availability

The requested project concept mentions budget, runtime, cast, director, popularity, and release season. The supplied dataset does not include these fields. The current baseline therefore uses only six predictors that are present. The optional enriched-training workflow requires a separate enriched CSV with the expected schema; none is currently included.

10.2 Forecasting Limitations

Rating and vote count are potentially post-release variables. Their use can make historical estimation look stronger than a genuine pre-release forecast. A valid pre-release model should use features available before release and evaluate on later release years. Revenue is also highly skewed and affected by external events not represented by the available fields.

10.3 Recommendation Interpretation

Graph links represent shared metadata. Flow and greedy algorithms are used to demonstrate graph and approximation concepts. The resulting recommendation is not a causal influence score, not a user-personalized recommender, and not evidence that a title will perform similarly at the box office.

10.4 Safe Academic Interpretation

Predictions are estimates from historical data and should be accompanied by limitations. They should not be used as guarantees or as the sole basis for investment, distribution, or employment decisions. Before submission, confirm department-specific declaration language and obtain the guide's approval and signatures.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Dataset and Modelling

- Add a documented, licensed enriched dataset containing budget, runtime, cast/director metadata, and release timing.
- Define prediction-time availability and remove post-release leakage.
- Evaluate time-based holdouts and compare Elastic Net with tree-based models.
- Report MAE, RMSE, R-squared, and prediction intervals where suitable.
- Track data and model versions, parameters, metrics, and environment details.

11.2 Application and Operations

- Add schema validation and clear missing-input feedback.
- Expose a documented REST endpoint using a lightweight serving framework if deployment is required.
- Monitor latency, input drift, prediction distributions, and delayed outcome performance.
- Add automated checks for cleaning, algorithms, pipeline loading, and UI-critical flows.
- Improve large-catalogue indexing and cache repeated analytics where profiling supports it.

CHAPTER 12: CONCLUSION

12.1 Summary

The project connects movie-data exploration, a supervised Elastic Net regression pipeline, a Streamlit application, and classical DSA demonstrations. Its source tree makes the workflow inspectable from CSV ingestion and cleaning through feature preparation, model training, prediction, evaluation, and user-facing analytics.

12.2 What Was Achieved

The implementation demonstrates title search with exact and approximate matching, metadata-based recommendations, graph algorithms, and a revenue-estimation model. It also documents the distinction between the currently supported baseline and the proposed enriched model, making the feature boundary visible rather than overstating the available data.

12.3 Learning Outcomes

The work provides practice with data quality, feature engineering, regularized regression, training-serving consistency, evaluation discipline, string algorithms, dynamic programming, graph flow, and responsible communication of model limitations. A future version can improve forecasting validity by adding point-in-time features, enriched data, and time-based testing.

CHAPTER 13: REFERENCES

- Scikit-learn documentation: ElasticNet, preprocessing, pipelines, model selection, and regression metrics. https://scikit-learn.org/stable/
- Streamlit documentation: application development and deployment. https://docs.streamlit.io/
- pandas documentation: data loading and dataframe operations. https://pandas.pydata.org/docs/
- NumPy documentation: numerical computing. https://numpy.org/doc/
- Python documentation. https://docs.python.org/3/
- Oracle Java documentation. https://docs.oracle.com/en/java/
- Knuth, D. E., Morris, J. H., and Pratt, V. R. Fast pattern matching in strings. SIAM Journal on Computing, 1977.
- Rabin, M. O. and Karp, R. M. Efficient randomized pattern-matching algorithms. IBM Journal of Research and Development, 1987.
- Levenshtein, V. I. Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics Doklady, 1966.
- The BoxOfficeRevenuePrediction project source files: README.md, PROJECT_DOCUMENTATION.md, PROJECT_REPORT.md, src/, java/, data/, and models/.

CHAPTER 14: APPENDICES

Appendix A. Project File Map

The project is organized around a root Streamlit launcher, a Python source package, Java algorithm source, raw and processed data, saved models, graphs, and documentation. Important modules include src/data_schema.py, src/clean_data.py, src/prepare_features.py, src/train_model.py, src/tune_model.py, src/evaluate_model.py, src/predict_revenue.py, and src/movie_dsa.py.

Appendix B. Installation and Setup

1. Open PowerShell and change to the BoxOfficeRevenuePrediction project folder.

2. If the existing virtual environment works, run the command shown in Chapter 9. If not, create a separate environment named .venv.

3. Install packages from requirements.txt in the selected environment.

4. Start the Streamlit application and open the local URL printed in the terminal (normally http://localhost:8501).

5. Keep the terminal open while using the local application; stop the server with Ctrl+C.

```
cd C:\Users\gopal\OneDrive\Desktop\BoxOfficeRevenuePrediction
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m streamlit run .\app\app.py
```

Appendix C. Key Terms

- **Elastic Net:** linear regression with combined L1 and L2 regularization.
- **Training-serving skew:** mismatch between transformations used to train a model and those applied when predicting.
- **Target leakage:** use of information unavailable at the intended prediction time.
- **One-hot encoding:** conversion of categories into indicator columns.
- **MAE / RMSE:** mean absolute error / root mean squared error.
- **Maximum flow:** greatest total flow from a graph source to a sink under edge capacities.

Appendix D. Academic Completion Checklist

- Confirm the guide designation and department.
- Confirm institution, programme, course code, section, team designation, and academic year.
- Obtain guide/coordinator/HOD signatures as required.
- Run the model evaluation workflow and insert verified metrics if requested.
- Add screenshots from the running application if required by the department.
- Verify citation style and declaration wording against institutional rules.